一、实验题目

基于Kaggle - Housing Prices Dataset数据集，分别使用Scikit-learn中的传统机器学习方法RandomForestRegressor及相关树类与简单DNN方法（Multilayer Perceptron, MLP）完成对房价的估值与趋势分析预判。对MLP设置多组超参数调试模型，探究不同超参数对模型性能的影响规律，并从预测精度对传统机器学习模型与MLP深度学习模型进行对比分析。

二、解题思路

（一）数据处理

1. 数据检视：确认数据集维度为(545, 13)，目标变量为price，其余12个为特征变量；通过统计明确变量类型，其中price、area、bedrooms等为数值型，mainroad、guestroom等为非数值型分类特征；经缺失值统计确认所有字段缺失值为0，无需进行缺失值填充处理。

2. Xy划分：将数据集拆分为训练集与测试集：以price作为目标变量y，其余12个字段（包含数值型与非数值型特征）共同构成特征集X。

3. 训练集测试集分割：使用train_test_split将X与y按7:3的比例划分为训练集与测试集；训练集用于模型参数学习与超参数调试，测试集使用训练集拟合的编码器与标准化器进行预处理，避免数据泄露与结果偶然性。

4. 非数值数据编码：对非数值型分类特征进行编码处理：均采用OneHotEncoder独热编码，编码产生的稀疏矩阵直接覆盖原非数值列；MLP和SVM模型对数据量纲差异敏感，对数值型特征执行StandardScaler标准化，消除特征间量纲差异，随机森林模型虽对特征尺度不敏感，仍统一纳入预处理流程以保证实验一致性。

（二）实例化模型

1. MLP模型参数设置：围绕隐藏层层数与结构、激活函数、最大训练轮数三个核心维度设计多组超参数组合，寻求最佳超参数设置。

2. RF：围绕树的数量（n_estimators）进行多组对比实验：设置50、100、200、300四组基学习器数量，观察模型性能随树数量的变化趋势。

3. SVM：调整核函数（kernel）进行多组对比实验：选择linear（线性核）、poly（多项式核）、rbf（径向基核）三类核心函数，分别对应不同的特征映射逻辑，对比三类核函数在房价回归任务中的拟合效果与泛化能力，找到最适配该数据集的核函数类型。

（三）拟合模型

将经过编码与标准化处理的训练集特征，分别输入三组模型的多组参数配置中进行拟合。

（四）预测评估

在测试集上对所有模型及参数组合生成房价预测结果，采用回归任务核心指标评估性能：均方根误差（RMSE）衡量预测值与真实值的平均偏差，平均绝对误差（MAE）衡量绝对偏差的平均水平，决定系数R^2衡量模型对房价方差的解释程度。

（五）最佳参数设置

以测试集R^2最高与RMSE最低为评价标准，筛选出各模型的最优参数组合。

三、实验细节

（一）One-Hot编码

```
num_features = df[numeric_cols]
cat_features = df[cat_cols]

# 初始化编码器
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
encoder.fit(X_train[cat_cols])

# 使用编码器转换
X_train_encoded = encoder.transform(X_train[cat_cols])
X_test_encoded = encoder.transform(X_test[cat_cols])

# 输出编码结果
onehot_cols = encoder.get_feature_names_out(cat_cols)
X_train_encoded_df = pd.DataFrame(X_train_encoded, columns=onehot_cols, inde
x=X_train.index)
```

（二）数据标准化

```
X_scaler = StandardScaler()
X_scaler.fit(X_train[numeric_cols])

# 去除One-Hot编码后的非数值特征，保留数值特征进行标准化
X_train_scaled = X_scaler.transform(X_train[numeric_cols])
X_test_scaled = X_scaler.transform(X_test[numeric_cols])

# 合并数据
X_train_processed = np.hstack([X_train_scaled, X_train_encoded])
X_test_processed = np.hstack([X_test_scaled, X_test_encoded])
```

（三）MLP模型自定义

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim1=128, hidden_dim2=64, activatio
n=nn.ReLU()):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim1)
        self.activation1 = activation
        self.fc2 = nn.Linear(hidden_dim1, hidden_dim2)
        self.activation2 = activation
        self.fc3 = nn.Linear(hidden_dim2, 1)

    def forward(self, x):
        hidden1 = self.fc1(x)
        hidden1_activated = self.activation1(hidden1)
        hidden2 = self.fc2(hidden1_activated)
        hidden2_activated = self.activation2(hidden2)
        output = self.fc3(hidden2_activated)
        output_activated = output.squeeze(1)
        return output_activated

def train_mlp(model, name, X_train_tensor, y_train_tensor,
              criterion = nn.MSELoss(), epochs=200):
    optimizer = torch.optim.Adam(model.parameters(), lr=0.005)
    loss_list =[]
    print(f"开始训练{name}模型...")
    for epoch in range(epochs):
        model.train()
        preds = model(X_train_tensor)
        loss = criterion(preds, y_train_tensor)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        loss_list.append(loss.item())
        if (epoch + 1) % 10 == 0:
            print(f"Epoch[{epoch + 1}/{epochs}], Loss: {loss.item():.4f}")
    return model, loss_list

input_dim = X_train_tensor.shape[1]
```

（四）MLP模型超参数设置

```
mlp_model_1 = MLP(input_dim, hidden_dim1=128, hidden_dim2=64, activation=nn.
ReLU())
criterion_1 = nn.MSELoss()
trained_model_1, loss_history_1 = train_mlp(
    model = mlp_model_1,
    name = " MLP-ReLU+MSE+128+64 ",
    X_train_tensor = X_train_tensor,
    y_train_tensor = y_train_tensor,
    criterion = criterion_1,
    epochs = 200)

mlp_model_1.eval()
with torch.no_grad():
    y_pred_mlp1 = mlp_model_1(X_test_tensor).numpy()
```

模型1：ReLU激活 + 双隐层（128+64）

模型2：Tanh激活 + 双隐层（128+64）

模型3：ReLU激活 + 单隐层（128）

模型4：ReLU激活 + 三隐层（128+64+32）

（五）RF模型超参数设置

模型实例化示例：

```
rf = RandomForestRegressor(n_estimators=200, random_state=42)
```

n_estimators测试：50、100、200、300

（六）SVM超参数设置

模型实例化示例：

```
svm = SVR(C=100, kernel='rbf')
```

kernel测试：linear、poly、rbf

四、结果分析

（一）模型最佳参数

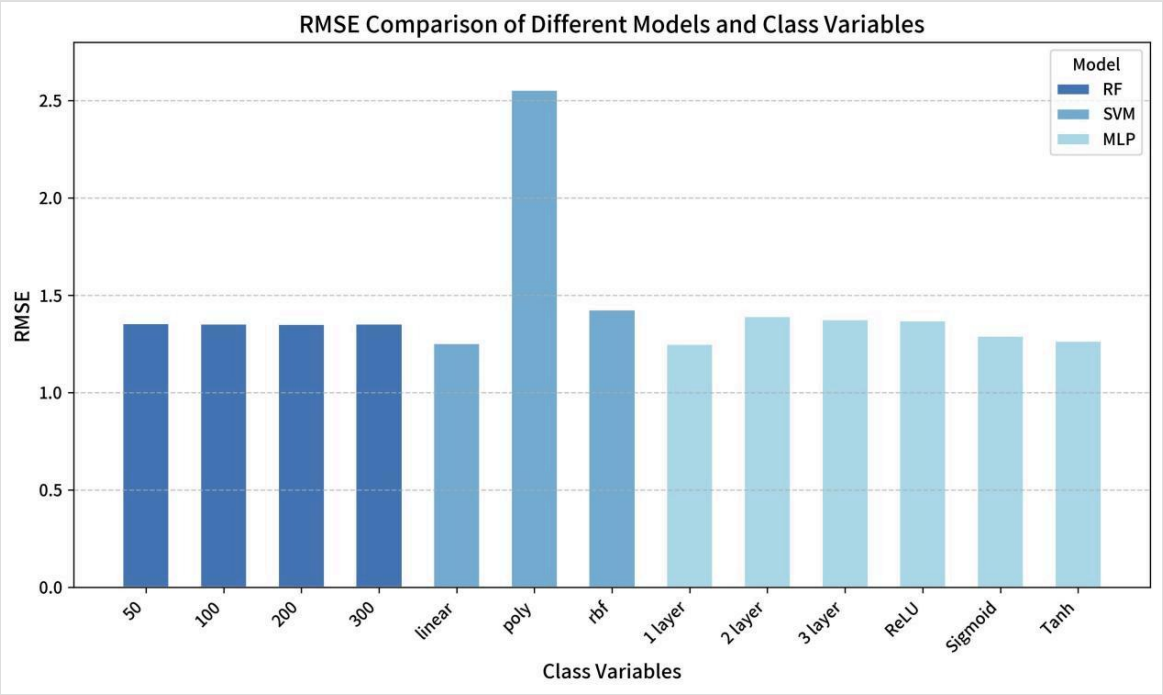

图1 各模型最优参数对比

随机森林模型在房价预测任务中，200棵树是最优设置。这是因为房价数据特征多而训练数据较少，过少的树会导致模型拟合能力不足，无法充分捕捉房价与特征之间的关联；而过多的树则会增加模型过拟合风险，200棵树的规模恰好平衡了拟合能力与泛化能力。

支持向量机模型测试了linear、poly、rbf三种核函数，linear核是房价预测的最优选择。说明房价数据的特征与房价的关联呈现出近似线性的趋势。

MLP模型中1层隐藏层是房价预测的最优设置。本次房价预测的训练数据对于DNN模型训练较少，过深的网络结构过拟合风险大，1层隐藏层足以捕捉数据中的非线性模式，且参数数量少，不易发生过拟合；2层、3层隐藏层增加了网络复杂度，导致模型泛化能力下降，预测误差显著升高。

房价数据的特征值分布范围较广，Tanh函数的输出范围为(-1,1)，能更好地标准化特征分布，缓解梯度消失问题；ReLU函数易出现“神经元死亡”现象，导致部分房价特征的信息无法有效传递；Sigmoid函数存在梯度饱和问题，且输出均值非0，影响模型收敛效率。

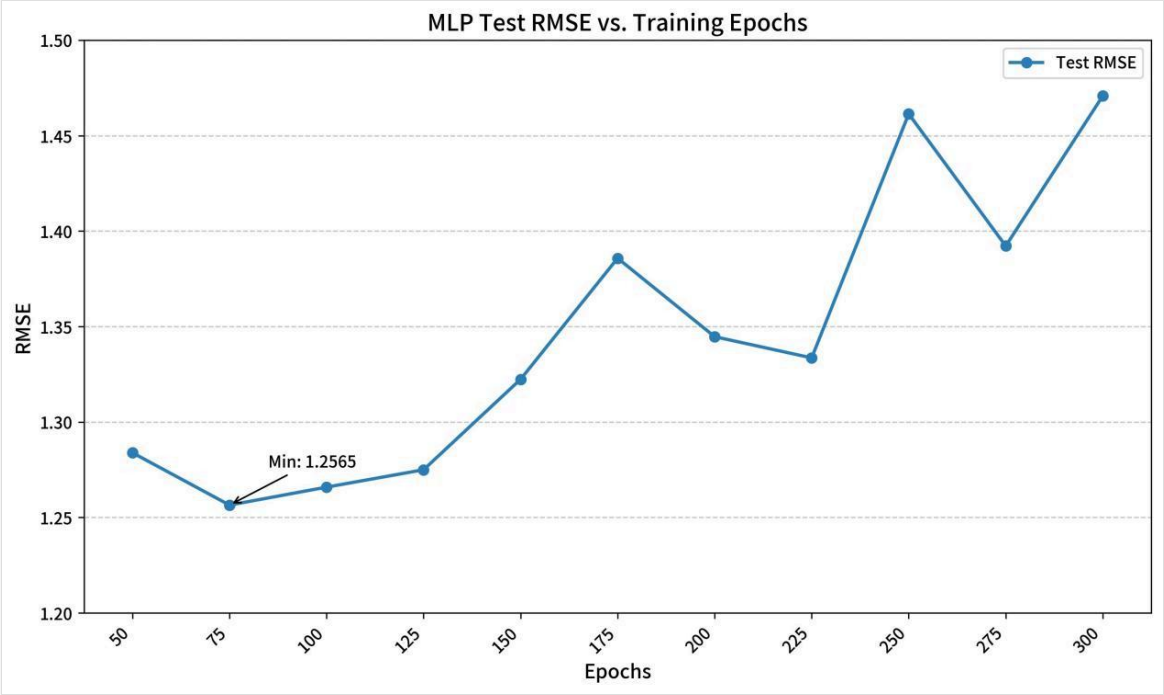

图2 MLP Test RMSE vs. Training Epochs

模型于75轮时达到最低值1.2565，75-125轮之间为稳定区间，之后误差明显增加，尤其在250轮后出现明显波动与大幅抬升。这说明75-125轮是该模型的最佳训练区间，泛化能力最优；之后测试集误差持续扩大，因此选择75-125轮的训练轮次区间既能保证模型拟合效果，又能有效避免过拟合风险。

（二）最佳模型

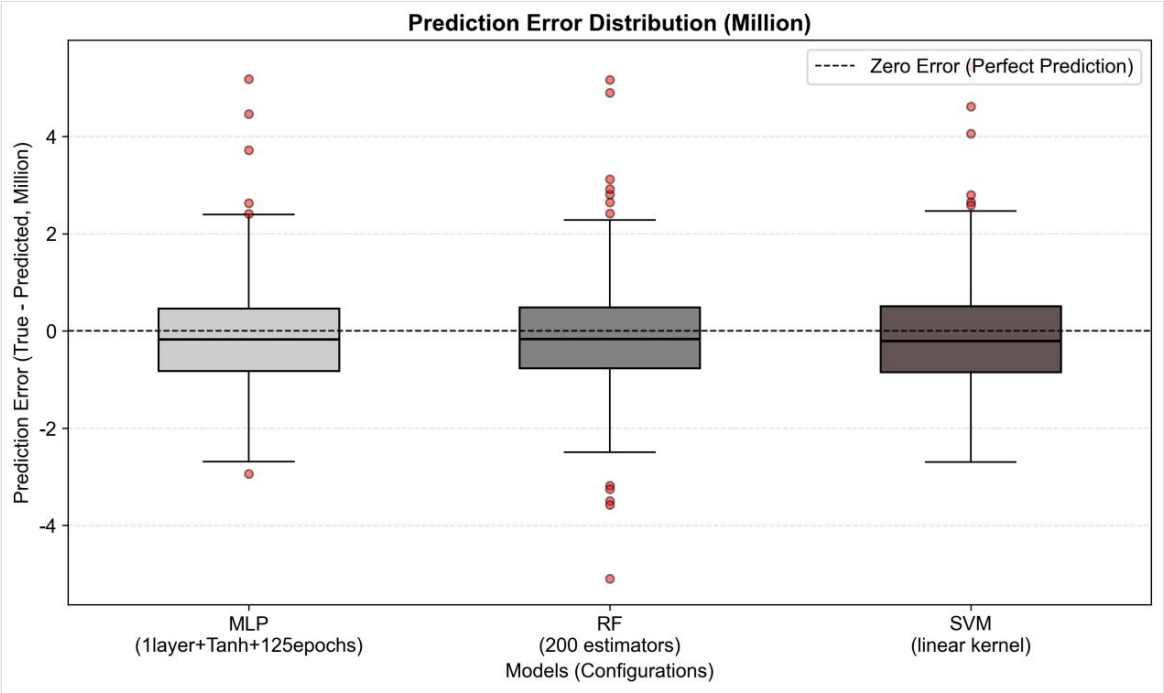

图3 模型预测误差分布箱线图

箱线图中箱体代表50%核心预测误差的四分位距，箱体越窄模型预测越稳定，中线为误差中位数（越接近0则整体偏差越小），须线为常规误差波动范围，红点为极端预测偏差，0误差虚线为完美预测基准线；结合RMSE、MAE与R^2指标来看，MLP的RMSE（1.228）和MAE（0.907）均为三者最低，R^2（0.650）最高，同时箱线图中MLP的箱体最窄、异常值最少，说明其预测最稳定且整体误差最小，因此MLP是本次实验的最佳模型。

五、个人心得

Q1:传统机器学习与深度学习方法区别？

传统机器学习依赖人工提取特征，模型结构相对简单，更适用于小规模、特征可解释的数据集；深度学习则通过多层神经网络自动学习特征（从底层浅层特征到高层抽象特征），模型参数规模大，需要大规模、高维度、非线性强的数据集。

Q2:训练集、测试集、验证集三集作用？

训练集用于模型的参数学习与拟合，是模型学习规律的核心数据，占比通常最大（60%-70%）；验证集用于超参数调优和模型选择，通过验证集的性能反馈调整模型配置，避免直接用测试集调参导致的过拟合；测试集是完全独立于训练与验证过程的未知数据，用于客观评估最终模型的泛化能力，模拟模型在真实场景中的表现，确保评估结果的公正性。

Q3:为什么先做数据划分再做数据清洗？

先划分数据集再做数据清洗是为了避免数据泄露，保证模型泛化能力评估的真实性：若先清洗数据（如填充缺失值、标准化、异常值处理）再划分，会导致测试集与验证集间接获取了训练集的信息。